Batch Pipeline in Docker

MySQL > Spark > Delta Lake > Postgres > dbt > Superset

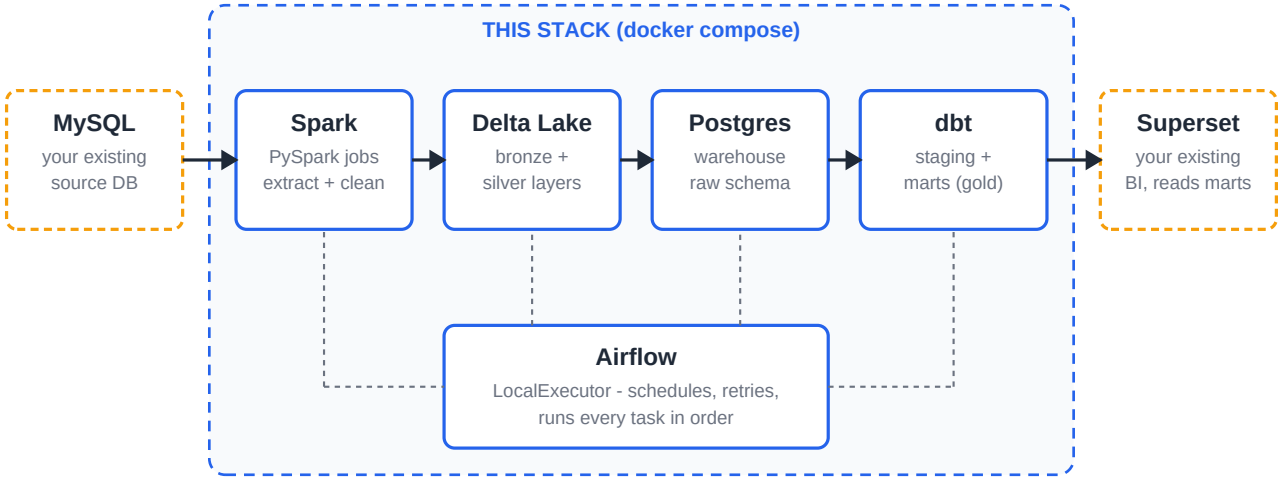
Setup, configuration and operations guide

Project folder: `batch-pipeline/`

1. Overview

This project runs a daily batch pipeline in Docker. It copies tables from an existing MySQL or MariaDB database into a Delta Lake, cleans them, loads them into a Postgres warehouse, and builds reporting tables with dbt. Your existing Superset then reads those reporting tables.

MySQL and Superset are **not** part of this stack. You point the pipeline at them through the `.env` file.



Dashed orange boxes are your existing services. Everything inside the blue boundary is started by `docker compose`.

Medallion layers

Layer	Stored in	Contents
Bronze	Delta Lake (`lake` volume)	Raw MySQL extracts, append-only, each row tagged with the Airflow run that loaded it.
Silver	Delta Lake (`lake` volume)	Lower-cased column names, trimmed strings, one row per primary key (latest version).
Raw	Postgres `warehouse.raw`	A copy of silver, so dbt can read it.
Gold	Postgres `warehouse.marts`	dbt models and tests. This is what Superset reads.

Containers

Service	Role
postgres	Warehouse database and Airflow metadata database. Published on host port 5433.
airflow-init	One-off: migrates the Airflow database and creates the admin user, then exits.
airflow-webserver	Airflow UI on port 8080.
airflow-scheduler	Schedules the DAG and runs every task. Spark (local mode) and dbt run here.

The Airflow services all use one custom image built from the `Dockerfile`. It contains Airflow 2.10, Java 17, PySpark 3.5 with Delta Lake 3.3, the MySQL and Postgres JDBC drivers, and dbt-postgres 1.9. Spark and dbt each have their own Python environment so their dependencies never conflict with Airflow's. The image builds for both amd64 and arm64.

2. Project layout

```
batch-pipeline/
  Dockerfile                custom image: Airflow + Spark + Delta + dbt
  docker-compose.yml        the stack (postgres + Airflow services)
  .env.example              settings template, copy to .env
  postgres/init/
    01-warehouse.sh         creates databases, schemas, read-only Superset user
  pipeline/                 mounted into the containers at /opt/pipeline
    config/tables.yml       which MySQL tables to extract
    dags/batch_pipeline.py  the Airflow DAG
    spark_jobs/
      common.py             Spark session, config and connection helpers
      bronze.py             MySQL -> Delta bronze
      silver.py             bronze -> silver
      load_warehouse.py     silver -> Postgres raw
    dbt/                    dbt project (staging views, mart tables, tests)
  docs/                     this document
```

3. Requirements

- Docker Engine 20.10 or newer with Compose v2 (Linux), or Docker Desktop (macOS, Windows).
- About 6 GB of RAM available to Docker. Spark's driver uses 2 GB by default (`SPARK_DRIVER_MEMORY`).
- Network access from Docker to your MySQL server.
- Internet access during the image build only. Nothing is downloaded at runtime.

4. Quick start

```
cp .env.example .env          # fill in MySQL credentials and passwords
# edit pipeline/config/tables.yml and the dbt models for your tables
docker compose up -d --build
```

- Open Airflow at `http://localhost:8080` and log in with `AIRFLOW_ADMIN_USER` / `AIRFLOW_ADMIN_PASSWORD`.
- Unpause the **batch_pipeline** DAG, or press the trigger button to run it now.
- After a successful run, the reporting tables are in `warehouse.marts`.

The included dbt models are examples written for `customers` and `orders` tables. If your source tables are different, replace the models first, or the `dbt_build` task will fail.

5. Configuration (.env)

Variable	Default	Purpose
MYSQL_HOST	host.docker.internal	Source database host. The default means the machine running Docker.
MYSQL_PORT	3306	Source database port.
MYSQL_DATABASE	-	Source database (schema) name.
MYSQL_USER / MYSQL_PASSWORD	-	A read-only user on the source.
MYSQL_JDBC_PARAMS	useSSL=false&...	Extra JDBC options. The default converts zero dates to NULL and uses UTC.
POSTGRES_USER / POSTGRES_PASSWORD	-	Warehouse owner. Avoid @ : / in the password.
SUPERSET_RO_PASSWORD	-	Password for the read-only superset_ro user.
WAREHOUSE_HOST_PORT	5433	Host port Superset uses to reach Postgres.
AIRFLOW_PORT	8080	Host port for the Airflow UI.
AIRFLOW_ADMIN_USER / _PASSWORD	admin / -	Airflow login created on first start.
AIRFLOW_SECRET_KEY	-	Long random string, e.g. openssl rand -hex 32.
AIRFLOW_FERNET_KEY	empty	Optional. Encrypts connections stored in Airflow.
PIPELINE_SCHEDULE	@daily	Airflow schedule (preset or cron expression).
SPARK_DRIVER_MEMORY	2g	Memory for each Spark job.
PIPELINE_IMAGE	batch-pipeline:latest	Image tag. Set to a registry tag to pull a prebuilt image.

6. Choosing tables (tables.yml)

`pipeline/config/tables.yml` lists every table the pipeline extracts. Each entry supports these keys:

Key	Required	Meaning
name	yes	Name used in the lake and in Postgres raw. Lowercase letters, digits and underscores only.
source	no	MySQL table name, if different from name (for example a table with spaces).
primary_key	no	Column, or list of columns, used to keep one row per record in silver. Without it, silver only removes exact duplicate rows.
incremental_column	no	A timestamp or increasing number. Each run only pulls rows where this value is at or above the highest value already loaded. Without it, the full table is pulled every run.
columns	no	List of columns to extract. Defaults to all columns.

```
tables:
  - name: orders
    primary_key: id
    incremental_column: updated_at

  # ERPNext / Frappe doctype
  - name: sales_invoice
    source: "tabSales Invoice"
    primary_key: name
    incremental_column: modified
    columns: [name, customer, posting_date, grand_total, status, docstatus, modified]
```

Changes to `tables.yml`, the DAG, the Spark jobs or the dbt models take effect on the next run without rebuilding, because `./pipeline` is mounted into the containers.

7. Connecting your MySQL

Create a read-only user on the source database:

```
CREATE USER 'pipeline_ro'@'%' IDENTIFIED BY 'change-me';
GRANT SELECT ON shop.* TO 'pipeline_ro'@'%';
```

Where MySQL runs	Set MYSQL_HOST to
Same machine as Docker	`host.docker.internal`. On Linux, MySQL must listen on an address containers can reach: if `bind-address = 127.0.0.1`, change it to `0.0.0.0` (or the Docker bridge IP) and firewall the port.
Another Docker container	Its container name, after attaching both to a shared Docker network. Or use `host.docker.internal` with its published port.
Remote server	Its hostname or IP address.

MariaDB works with the bundled MySQL driver.

8. Connecting your Superset

In Superset, add a PostgreSQL database with this SQLAlchemy URI:

```
postgresql://superset_ro:<SUPERSET_RO_PASSWORD>@<docker-host>:5433/warehouse
```

- `<docker-host>` is `localhost` when Superset runs directly on the same machine, or `host.docker.internal` when Superset runs in its own container on that machine.
- `superset_ro` can only read the `marts` schema. New mart tables are granted to it automatically.

9. The pipeline (Airflow DAG)

The DAG **batch_pipeline** runs four tasks in order. Each task retries twice, five minutes apart, and only one run is active at a time.

Task	Script	What it does
mysql_to_bronze	spark_jobs/bronze.py	Reads each configured table over JDBC (incrementally when configured) and writes it to Delta bronze, tagged with the Airflow run id.
bronze_to_silver	spark_jobs/silver.py	Rebuilds silver from bronze: lower-cases column names, trims strings, keeps the latest row per primary key.

Task	Script	What it does
silver_to_warehouse	spark_jobs/load_warehouse.py	Truncates and reloads each `raw.<table>` in Postgres.
dbt_build	dbt build	Builds staging views and mart tables and runs all tests. A failing test fails the task.

10. dbt project

- **staging/**: one view per raw table that renames and casts columns (schema `staging`).
- **marts/**: reporting tables for Superset (schema `marts`). The examples are `dim_customers`, `fct_orders` and `daily_revenue`.
- **sources.yml** must list every raw table the models read.
- **schema.yml** files hold tests (unique, not_null, relationships).
- The `generate_schema_name` macro makes models land in `staging` and `marts` exactly, instead of dbt's default prefixed names.

To add a table: add it to `tables.yml`, add it to `sources.yml`, write a `stg_` model, then build marts on top of it.

11. Reruns and data safety

- **Bronze**: a rerun of the same Airflow run replaces only that run's rows, so retries never duplicate data.
- **Incremental loads** start from the highest value loaded by earlier runs. They use >=, so rows sharing the boundary timestamp are not missed, and silver removes the overlap.
- **Silver and raw** are rebuilt from bronze on every run, so they are always consistent.

12. Operations

Run a single step by hand

```
docker compose exec airflow-scheduler bash -c \
  "cd /opt/pipeline/spark_jobs && /opt/spark-venv/bin/python silver.py"

docker compose exec airflow-scheduler \
  /opt/dbt-venv/bin/dbt build --project-dir /opt/pipeline/dbt --profiles-dir /opt/pipeline/dbt
```

Ship the image to another machine

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t <registry>/batch-pipeline:1.0 --push .
```

On the target machine, copy `docker-compose.yml`, `.env`, `postgres/` and `pipeline/`. Set `PIPELINE_IMAGE` to the registry tag and run `docker compose up -d --no-build`. To run only from the code inside the image, remove the `./pipeline` mount from the compose file.

Stop and reset

```
docker compose down        # stop, keep all data
docker compose down -v     # stop and DELETE lake, warehouse and Airflow metadata
```

The Postgres init script runs only on the first start with an empty data volume. Changing `SUPERSET_RO_PASSWORD` later needs an `ALTER ROLE` or a reset.

13. Troubleshooting

Symptom	Likely cause and fix
mysql_to_bronze fails with "Communications link failure"	Containers cannot reach MySQL. Check MYSQL_HOST, the MySQL bind-address, firewall rules, and that the user is allowed from a remote host ('user'@'%').
Access denied for user	Wrong credentials, or the user lacks SELECT on the database.
silver_to_warehouse fails after a source table changed	Raw tables are truncated, not recreated, so new or removed columns do not fit. Run `DROP TABLE raw.<name> CASCADE;` once, then re-run. dbt recreates its views.
dbt_build fails on a model	The example models do not match your tables. Update sources.yml and the models.
dbt_build fails on a test	The data broke a rule (for example a duplicate id). Models that depend on the failing one are skipped. The task log names the failing test.
Spark task killed / exit code 137	Out of memory. Give Docker more RAM or lower SPARK_DRIVER_MEMORY.
Airflow UI login fails on first start	airflow-init did not finish. Check `docker compose logs airflow-init`.
Superset cannot connect	Use port 5433 (WAREHOUSE_HOST_PORT), database warehouse, user superset_ro, and the right host (localhost or host.docker.internal).

14. Known limits

- Rows deleted in MySQL are not detected. Use soft-delete flags, or CDC (for example Debezium) if you need deletes.
- Spark runs on a single node in local mode. That suits up to tens of millions of rows. Beyond that, move Spark to a cluster.
- Large tables are read through a single JDBC connection. Partitioned reads can be added in `bronze.py` if extraction becomes slow.